Relazione Progetto di Reti Logiche 2024-2025

Autori:
Daniele Confalonieri
Andrea Compagnoni

2024/2025

1 Introduzione

Il progetto implementato realizza, in VHDL, il modulo `project_reti_logiche` descritto nella specifica della Prova Finale di Reti Logiche dell'anno accademico 2024-2025. Il funzionamento di tale modulo consiste nel leggere da memoria (a partire da un indirizzo di base `ADD`) un blocco di 17 byte che definisce la lunghezza della sequenza da processare, la scelta del tipo di filtro e i coefficienti usati nel processo di filtraggio, quindi applicare un *filtro differenziale* sulle K parole (ogni parola è un byte in complemento a 2, nel range $[-128, 127]$) e riscrivere la sequenza filtrata $\{R[i]\}_{i=0}^{K-1}$ in memoria a partire dall'indirizzo $\texttt{ADD} + 17 + K$.

I dati letti dal blocco iniziale sono:

- `K1`, `K2` (2 byte) che definiscono la lunghezza K (K1 byte più significativo, K2 byte meno significativo);
- `S` (1 byte) il selettore del filtro: bit meno significativo = 0 $\Rightarrow$ filtro ordine 3, =1 $\Rightarrow$ filtro ordine 5;
- 14 byte $\{C_j\}$ che contengono i coefficienti dei due filtri (posizionati come specificato nella specifica).

L'operazione di filtraggio è data da

$$R[i] = \frac{1}{n} \sum_{j=-\ell}^{\ell} C_j \cdot W[i+j]$$

con $\ell = 2$ per il filtro di ordine 3 e $\ell = 3$ per il filtro di ordine 5; La normalizzazione numerica è stata pensata con la tecnica shift-add (approssimazione per potenze di due), con correzione dei troncamenti per valori negativi (metodo che riduce l'errore sistematico introdotto dagli shift), n è il fattore di normalizzazione.

Altre condizioni imposte:

- i valori risultanti vanno saturati in $[-128, 127]$ prima della scrittura;
- le parole fuori dalla finestra di inizio e fine sequenza sono considerate nulle (ad esempio: $W[-1] = 0, W[K+1] = 0$);

- l'interfaccia è sincrona sul fronte di salita del clock, tranne `i_rst` che è asincrono.

L'implementazione ha come obiettivi principali: correttezza funzionale rispetto alla specifica, robustezza (gestione corretta del protocollo START/DONE/RESET e una scelta conservativa delle ampiezze delle variabili per evitare overflow durante i calcoli) e semplicità di verifica (chiare separazioni tra circuito sequenziale e combinatorio).

Segnali di input e output Il modulo definito da `project_reti_logiche` si collega alla memoria e al sistema esterno mediante i seguenti segnali:

`i_clk` **(in)** Clock sincrono: tutte le operazioni – dalla FSM alla logica combinatoria – avvengono sul fronte di salita.

`i_rst` **(in)** Reset asincrono: riporta immediatamente lo stato interno a `RESET`, reinizializzando i contatori, registri di indirizzo, risultati e segnali di controllo.

`i_start` **(in)** Segnale di avvio: quando è alto, la FSM lascia lo stato `RESET` e inizia la lettura del blocco iniziale (K1, K2, selettore filtro, coefficienti). Solo quando torna basso dopo il termine elaborazione, il modulo si riconfigura per un nuovo ciclo.

`i_add` **(in)** Indirizzo base (ADD): definisce il punto di partenza in memoria da cui leggere i parametri e iniziare la memorizzazione dei risultati.

`i_mem_data` **(in)** Dati letti dalla memoria: registro in cui vengono memorizzate le parole lette (e scritte) dalla memoria, viene aggiornato se `o_mem_we`=1.

`o_mem_addr` **(out)** Indirizzo di memoria: il circuito aggiorna questo segnale prima di ogni accesso, sia in lettura che in scrittura, seguendo l'avanzamento del indirizzo di lettura o scrittura corrente.

`o_mem_en` **(out)** Enable memoria: alto quando si richiede un accesso alla memoria; basso nei cicli di elaborazione interna in cui non serve.

`o_mem_we` **(out)** Write-enable: valore logico alto solo quando si vuole scrivere dati (nella fase `WRITE_RESULT`), altrimenti a 0 per le letture.

`o_mem_data` **(out)** Dati da scrivere in memoria: validi solo quando `o_mem_we`=1; contiene il risultato saturato e normalizzato del filtro.

`o_done` **(out)** Segnale di completamento: viene portato alto in `TERMINATE`, per indicare che l'elaborazione è finita. Resta alto fino a che `i_start` non torna zero.

Questo protocollo assicura:

- una chiara fase iniziale di configurazione (reset + start),
- una precisa temporizzazione degli accessi alla memoria (tramite `ENABLE` + `WE`),
- la sincronizzazione tra indirizzi e dati letti/scritti,
- una segnalazione definitiva di fine ciclo per il sistema esterno.

Esempio numerico Per chiarire il funzionamento, consideriamo una sequenza di 5 valori memorizzati in complemento a 2 a partire dall'indirizzo `ADD+17`:

$$W = \{\, 1,\ 2,\ 3,\ 4,\ 5 \,\}$$

e il filtro di ordine 3, i cui coefficienti sono:

$$c = [0, -1, 8, 0, -8, 1, 0], \quad n = 12.$$

Il calcolo del primo valore filtrato $R[0]$ utilizza due parole "fittizie" pari a 0 (padding), quindi:

$$R[0] = \frac{1}{12}\big(-1 \cdot 0 + 8 \cdot 0 + 0 \cdot 1 - 8 \cdot 2 + 1 \cdot 3\big) = \frac{-13}{12}.$$

La normalizzazione è implementata con la decomposizione per shift:

$$\frac{1}{12} \approx \frac{1}{16} + \frac{1}{64} + \frac{1}{256} + \frac{1}{1024},$$

per cui il risultato finale si ottiene come somma di più shift-right del buffer. Nel caso in cui il valore sia negativo, ad ogni shift viene aggiunto 1 per correggere il troncamento. Applicando questa approssimazione, si ottiene:

$$R[0] \approx 0 \quad \text{(invece di } -1 \text{ esatto)}.$$

Procedendo con le altre parole, ogni valore viene calcolato come combinazione lineare pesata dei vicini, normalizzato e saturato in $[-128, 127]$. Ad esempio, il secondo valore risulta:

$$R[1] = \frac{1}{12}\big(-1 \cdot 0 + 8 \cdot 1 + 0 \cdot 2 - 8 \cdot 3 + 1 \cdot 4\big) = \frac{-12}{12} \approx 0.$$

L'intera sequenza R viene quindi scritta in memoria a partire da `ADD+17+K`.

2 Architettura della Rete

L'architettura logica è una classica separazione tra circuito sequenziale e combinatorio. Questo approccio rende più semplice: la verifica delle transizioni, l'analisi dei tempi e la manutenzione del codice. Nel codice VHDL fornito la separazione è esplicita in più processi: (i) aggiornamento degli stati sincrono con reset asincrono, (ii) calcolo del next-state, (iii) processo che esegue le operazioni combinatorie e aggiorna i registri dati, (iv) processo che assegna i segnali di controllo e di memoria in base allo stato.

2.1 Modulo Combinatorio

Il modulo combinatorio contiene i registri temporanei e la logica aritmetica:

`filters_reg` **(56 bit)** registro che concatena fino a 7 coefficienti da 8 bit ciascuno (7*8 = 56). Questo formato permette di gestire agevolmente sia il filtro di ordine 3 che quello di ordine 5 con la stessa struttura di shift (ogni nuovo coefficiente viene concatenato con uno `sll 8`).

`memorized_words_reg` **(56 bit)** registro che contiene le 7 parole correnti necessarie per calcolare il valore di R[i]. Per semplicità si usa la stessa ampiezza di 56 bit per gestire fino a 7 campioni collegati alla dimensione massima del filtro.

`result_buffer_reg` **(24 bit)** buffer per la somma pesata $\sum C_j \cdot W[\cdot]$. La scelta di 24 bit è volutamente conservativa: vediamo perché.

`reading_address_reg`: contiene l'indirizzo corrente da cui leggere dati in memoria. Viene inizializzato con il valore di `i_add` durante il reset e aggiornato progressivamente a ogni lettura (per esempio, dopo aver letto `K1` viene incrementato per accedere a `K2`, e così via). Nelle fasi di acquisizione dei coefficienti e delle parole W, questo registro viene incrementato di 1 o di più posizioni a seconda della tipologia di filtro selezionato (ordine 3 o 5).

`writing_address_reg`: contiene l'indirizzo in cui devono essere scritti i risultati filtrati R. È inizializzato a `i_add` $+ K + 17$, cioè subito dopo il blocco iniziale e le parole W. Dopo ogni scrittura di un byte filtrato, il registro viene incrementato per puntare alla cella successiva.

Giustificazione ampiezze — calcolo dimensione buffer Per dimensionare correttamente il registro `result_buffer_reg` è necessario considerare il caso peggiore in cui i coefficienti e i campioni assumono valori massimi in valore assoluto.

- I coefficienti C_j e i dati $W[i]$ sono rappresentati su 8 bit signed, quindi variano da -128 a $+127$.
- Se tutti i coefficienti sono pari a -128 e anche i campioni sono pari a -128, ciascun prodotto $C_j \cdot W[i]$ vale 16384.
- Per il filtro di ordine massimo (7 coefficienti, ordine 5) si hanno 7 prodotti contemporanei:

$$7 \times 16384 = 114688.$$

Questo valore deve essere rappresentato in complemento a 2. Poiché $2^{16} = 65536 < 114688 < 2^{17} = 131072$, occorrono almeno 17 bit per rappresentare il valore assoluto.

Considerando anche il segno, il numero minimo di bit signed è 18. Tuttavia, per semplicità e per rendere più agevoli le operazioni di normalizzazione e saturazione, si è scelto di utilizzare un registro da 24 bit (3 byte). Questo margine garantisce che non si verifichi overflow durante il calcolo del risultato, neanche nei casi estremi di valori massimi negativi o positivi, e permette di effettuare le operazioni di shift-add richieste dalla normalizzazione in totale sicurezza.

Assegnamento a `o_mem_addr` Il segnale di uscita `o_mem_addr` viene assegnato, in base allo stato della FSM, a uno dei due registri:

- durante le fasi di lettura (`ASK_MEM`, `READ_*`), `o_mem_addr` assume il valore di `reading_address_reg`;
- durante le fasi di scrittura (`WRITE_RESULT`, `NORMALIZE_RESULT`), `o_mem_addr` viene assegnato a `writing_address_reg`.

Questo meccanismo a doppio registro ha due vantaggi principali:

1. separa chiaramente il flusso di lettura da quello di scrittura, evitando conflitti o sovrascritture indesiderate;

2. riduce la logica combinatoria necessaria a calcolare ogni volta l'indirizzo corretto, poiché entrambi i registri vengono aggiornati in modo incrementale e indipendente.

In questo modo, l'FSM può gestire facilmente il ciclo di vita dei dati: lettura dei parametri e della sequenza in ingresso, elaborazione interna e scrittura sequenziale dei risultati nella regione di memoria dedicata.

Operazioni aritmetiche La moltiplicazione fra byte signed e coefficienti signed viene eseguita convertendo ciascun operando a `signed` e poi effettuando `resize(..., 24)` prima della somma per evitare overflow di tipo. Il buffer usa quindi operazioni `resize(...)` e `signed(...)` per mantenere la correttezza del segno.

Normalizzazione per shift-add La divisione per n è approssimata con shift-right e somma di potenze di due, come richiesto dalla specifica:

- ordine 3: $1/12 \approx 1/16 + 1/64 + 1/256 + 1/1024 \Rightarrow$ shift di 4,6,8,10 bit;
- ordine 5: $1/60 \approx 1/64 + 1/1024 \Rightarrow$ shift di 6 e 10 bit.

Per ridurre l'errore su numeri negativi si applica una correzione: ad ogni shift, se l'argomento è negativo si aggiunge 1 al risultato del singolo shift. Il codice somma queste correzioni cumulative (4 per l'ordine 3, 2 per l'ordine 5) quando il buffer è negativo, replicando esattamente il metodo della specifica.

Saturazione Dopo normalizzazione il risultato (a 24 bit) viene confrontato con i limiti a 8 bit signed. Se maggiore di +127 o minore di -128 viene saturato rispettivamente a +127 o -128 prima della scrittura su `o_mem_data`.

2.2 Modulo Sequenziale (FSM)

Il controllo è realizzato come un'automa a stati finiti (enumerazione VHDL) con stati principali: `RESET`, `START`, `ASK_MEM`, `WAIT_MEM`, `READ_K1`, `READ_K2`, `READ_T`, `READ_FILTER`, `SHIFT_FILTER`, `READ_WORD`, `SHIFT_WORD`, `CALC_RESULT`, `NORMALIZE_RESULT`, `WRITE_RESULT`, `TERMINATE`.

Ideazione e struttura della FSM Per capire meglio come funziona la macchina a stati, è utile guardare alcune transizioni che sono un po' il cuore del circuito.

- **RESET → START**: questa è la partenza. La FSM rimane ferma in RESET finché non arriva un `i_start=1`. Solo in quel momento passa a START e si mette in moto. È importante perché assicura che non parta mai da sola senza una richiesta esplicita.
- **WAIT_MEM → vari stati di lettura**: Questa sezione viene approfondita successivamente, in generale lo stato successivo rispetto a `WAIT` viene scelto in base a `prev_state` ed in alcuni casi in base ai registri e `writing_address_reg` e `reading_address_reg`, per capire a che punto si è dell'elaborazione dell'input.

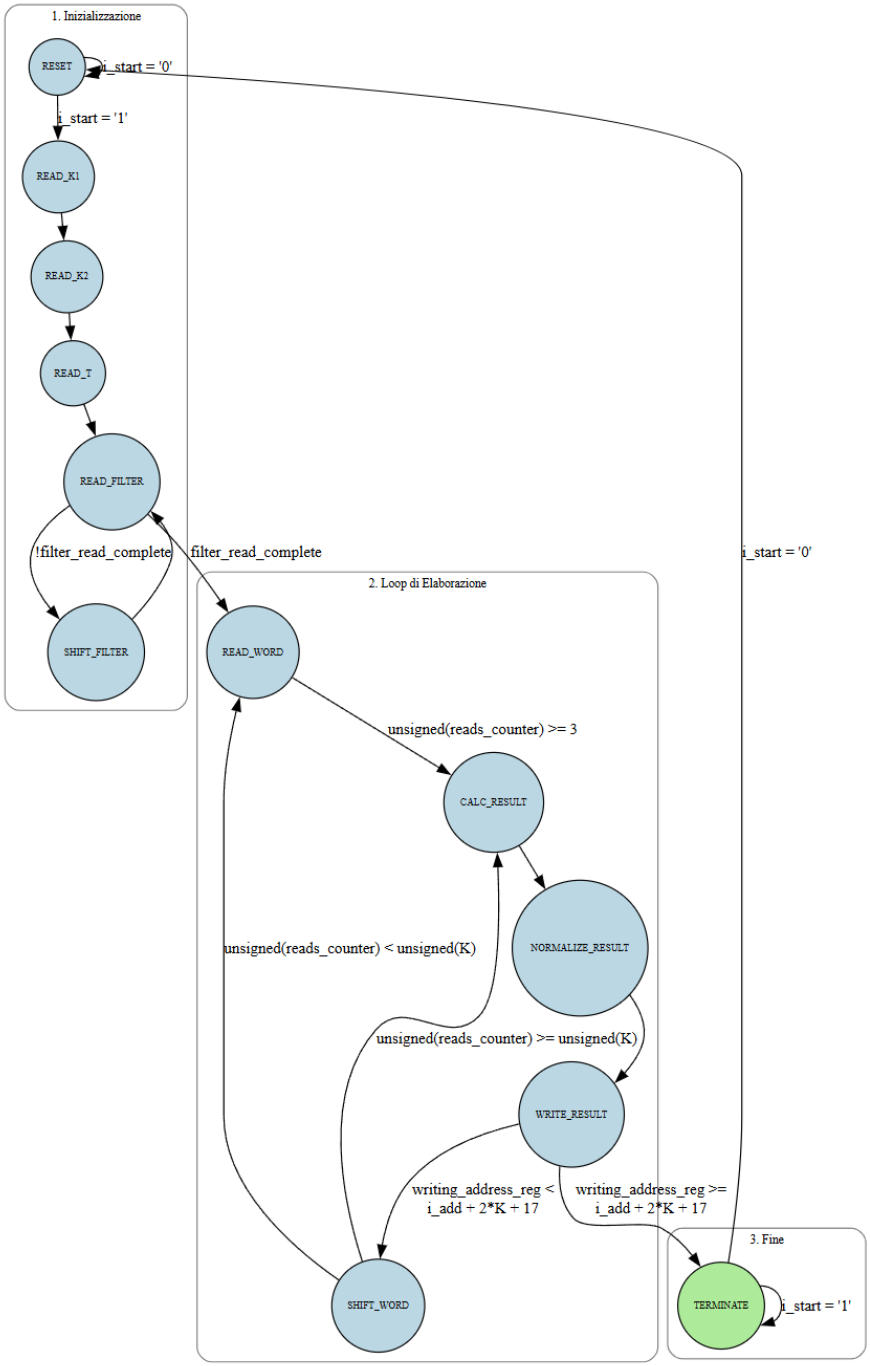


Figura 1: Diagramma della Macchina a Stati Finiti (FSM)

- **READ_FILTER → SHIFT_FILTER / READ_WORD**: qui il comportamento dipende da `typeof_filter_reg` e dall'indirizzo che stiamo leggendo. Se abbiamo ancora coefficienti da prendere, passiamo a `SHIFT_FILTER` per spostare quelli già letti e far spazio al successivo. Quando invece abbiamo finito, la FSM va a `READ_WORD` e inizia a lavorare sui campioni W.
- **READ_WORD → SHIFT_WORD → CALC_RESULT**: ogni volta che leggiamo una parola incrementiamo `reads_counter`. Finché non abbiamo letto abbastanza byte (meno di K), si resta nel ciclo lettura/shift. Quando invece `reads_counter`

ha raggiunto il valore K, significa che abbiamo accumulato abbastanza dati e quindi possiamo andare in `CALC_RESULT`.

- **NORMALIZE_RESULT → WRITE_RESULT**: qui non ci sono condizioni particolari, la transizione è sempre fatta. Ma è importante perché segna il passaggio dalla fase di calcolo "interno" a quella in cui il risultato diventa visibile verso la memoria. È il punto in cui il dato viene preparato (con saturazione e correzione) e reso pronto per essere scritto.
- **WRITE_RESULT → SHIFT_WORD / TERMINATE**: dipende dal valore di `writing_address_reg`. Se non abbiamo ancora scritto tutti i risultati, torniamo a `SHIFT_WORD` e ricominciamo il giro. Quando invece l'indirizzo di scrittura ha superato la soglia `i_add` $+ 2K + 17$, significa che la sequenza R è completa e si può passare a `TERMINATE`, alzando `o_done`.
- **TERMINATE → RESET**: avviene solo quando `i_start` torna basso. Questo evita che il modulo resti bloccato con DONE=1 e impedisce di partire con una nuova elaborazione senza prima aver segnalato la fine della precedente.

In poche parole, queste transizioni sono quelle che segnano i momenti chiave: partire, leggere i parametri, accumulare dati, calcolare, scrivere e infine dire al sistema "ho finito". Sono proprio queste le condizioni che fanno funzionare il tutto in modo ordinato.

Lettura da memoria La memoria nel testbench è un blocco RAM sincrono con latenza di lettura (read data disponibile nel ciclo successivo all'assert di `o_mem_en` con `o_mem_we`=0). Per gestire questo comportamento si usa una sequenza `ASK_MEM` (prepara indirizzo e abilita la memoria) seguita da `WAIT_MEM` (attesa della risposta). Per capire in che stato fare la transizione da `WAIT_MEM`, lo stato precedente è memorizzato in `prev_state` e quindi il next-state dipende dal valore di `prev_state`. Questo pattern evita ambiguità su quale lettura si sta ottenendo nella risposta della memoria. L'implementazione dell'interazione con la memoria del nostro modulo è coerente con il template fornito nel testbench.

Interazione con il modulo combinatorio Ogni stato della FSM attiva un segnale di controllo e eventualmente l'abilitazione alla lettura/scrittura: ogni stato della FSM ha il corrispondente segnalino, il modulo combinatorio "ascolta" questi segnali e, in base a quale è attivo, esegue l'operazione corrispondente (ad esempio, se `reading_k1` è attivo, carica il byte letto da memoria in `K(15 downto 8)` e incrementa l'indirizzo di lettura).

Meccanismo di transizione Il processo `fsm_state_logic` calcola il `next_state` in modo puramente combinatorio, tenendo conto dello `current_state`, di `i_start` e dei registri interni. Nel ciclo successivo di clock, il processo `fsm_update_state` aggiorna il `current_state` al valore di `next_state`. Contestualmente, il processo `changing_state` attiva il segnalino relativo allo stato in cui ci si trova, abilitando così la logica del circuito combinatorio.

Questo schema a tre processi (update, next-state, attivazione segnali) rende l'FSM modulare:

1. Le transizioni vengono calcolate sempre in anticipo e rese stabili prima del fronte di clock successivo.

2. I registri interni influenzano direttamente le condizioni di transizione, assicurando la coerenza tra logica combinatoria e sequenziale.

3. I "segnalini" garantiscono che in ogni stato venga eseguita una sola azione ben definita, evitando conflitti o sovrapposizioni.

3 Risultati sperimentali

3.1 Sintesi

1. Report Utilization

   Il report di utilizzo delle risorse è stato generato dallo strumento di sintesi Xilinx Vivado dopo le fasi di sintesi e implementazione. La tabella 1 riassume l'occupazione delle principali risorse logiche della FPGA target.

   L'utilizzo complessivo è molto basso (inferiore all'1%), a conferma del fatto che il design è efficiente e non intensivo dal punto di vista delle risorse.

   E' inoltre importante evidenziare l'assenza di Latch.

Tabella 1: Report di utilizzo delle risorse logiche dopo la sintesi.

Tipo di Risorsa	**Usate**	**Fisse**	**Disponibili**	**Utilizzo %**
Slice LUTs*	929	0	134600	0.69
LUT come Logica	929	0	134600	0.69
LUT come Memoria	0	0	46200	0.00
Slice Registers	242	0	269200	0.09
Registro come Flip Flop	242	0	269200	0.09
Registro come Latch	0	0	269200	0.00
F7 Muxes	0	0	67300	0.00
F8 Muxes	0	0	33650	0.00

2. Timing Report

   L'analisi statica dei tempi (Static Timing Analysis - STA) è stata eseguita per verificare che il circuito rispetti i vincoli temporali imposti da un clock con periodo di 20 ns (50 MHz). I risultati, riassunti in Tabella 2, confermano che il design è privo di violazioni di timing.

Tabella 2: Sommario dell'analisi dei tempi con clock a 50 MHz.

Metrica di Timing	**Slack Peggiore (ns)**	**Endpoint Falliti**
Setup (WNS)	4.805	0
Hold (WHS)	0.175	0
Pulse Width (WPWS)	9.500	0

I punti chiave emersi dal report sono:

- **Nessuna violazione di Setup:** Il *Worst Negative Slack* (WNS) è positivo e pari a **4.805 ns**. Questo indica che il percorso critico del circuito è quasi 5 ns più veloce del necessario, garantendo un ampio margine di sicurezza rispetto al periodo di clock di 20 ns.

Dato il WNS, possiamo calcolare il ritardo effettivo del percorso critico:

$$\text{Ritardo Percorso Critico} = \text{Periodo Clock} - \text{WNS} = 20\,\text{ns} - 4.805\,\text{ns} = 15.195\,\text{ns}$$

Di conseguenza, la frequenza massima teorica a cui il circuito può operare è:

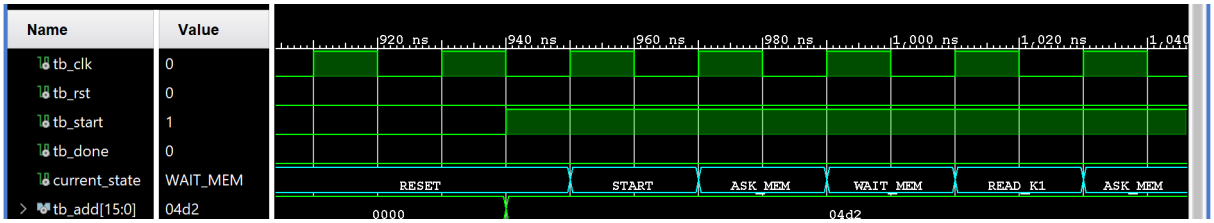
$$F_{\max} = \frac{1}{15.195\,\text{ns}} \approx 65.81\,\text{MHz}$$

Questo valore è superiore alla frequenza target di 50 MHz, dimostrando la robustezza del design dal punto di vista temporale e un buon margine di performance.

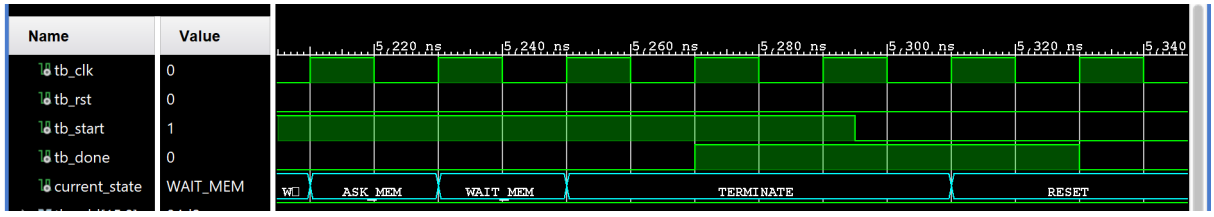
3.2 Simulazioni

3.3 Test bench prototipo

Il primo test bench è quello "standard". Questo test bench, molto semplice ci ha aiutato nelle prime fasi di progettazione, soprattutto durante la scrittura e la verifica del corretto funzionamento del codice. Di seguito sono state analizzate le forme d'onda generate dal test bench. Le figure 2a e 2b mostrano i momenti cruciali dell'avvio e della terminazione del processo.



(a) Comportamento all'avvio del processo.



(b) Comportamento in fase di terminazione.

Figura 2: Verifica del protocollo di comunicazione START/DONE tramite forme d'onda.

Fase di Avvio Come mostrato in Figura 2a, la FSM si trova inizialmente nello stato `RESET`. Quando il segnale di test bench `tb_start` (corrispondente all'input `i_start`) viene portato a livello logico alto, la macchina a stati innesca correttamente la transizione allo stato `START`. Da qui, inizia il ciclo di operazioni previsto dalla specifica, con la sequenza di stati `ASK_MEM`, `WAIT_MEM` e `READ_K1` per la lettura dei parametri iniziali dalla memoria.

Fase di Terminazione La Figura 2b illustra il completamento del ciclo. Dopo aver terminato l'elaborazione e la scrittura di tutti i risultati in memoria, la FSM entra nello stato `TERMINATE` e, di conseguenza, il segnale di output `o_done` viene asserito (portato a '1'). Il protocollo richiede che il test bench, una volta letto `o_done`='1', abbassi il segnale `i_start`. Si osserva che, non appena `i_start` torna a '0', la FSM esegue correttamente la transizione dallo stato `TERMINATE` allo stato `RESET`, rendendo il modulo pronto per una nuova elaborazione.

3.3.1 Test bench completo

Oltre al test bench standard, è stato utilizzato un test bench avanzato progettato specificamente per "stressare" il componente in condizioni operative critiche e verificarne la robustezza. Questo test bench convalida il design attraverso una serie di scenari complessi, come descritto di seguito.

- **Scenari Multipli e Consecutivi:** Il test bench esegue sei scenari di test in successione. Questo verifica la capacità della FSM di gestire correttamente il protocollo START/DONE, tornando a uno stato di attesa pulito e ripartendo per una nuova elaborazione non appena `i_start` viene riasserito (come spiegato sopra).
- **Test di Carico con Sequenze Lunghe:** Vengono testati sia il filtro di ordine 3 che quello di ordine 5 utilizzando sequenze di input molto lunghe (fino a $K = 12000$ parole). Questi test di lunga durata verificano la corretta gestione dei contatori interni e degli indirizzi di memoria su un vasto range di operazioni, assicurando l'assenza di errori di accumulazione o di overflow dei contatori.
- **Verifica dei Casi Limite (Corner Cases):** Sono stati inclusi scenari specifici per testare i limiti aritmetici del datapath:
  - Uno scenario utilizza come input una sequenza di soli valori massimi positivi (`+127`) e coefficienti massimi negativi (`-128`).
  - Un altro scenario testa il caso opposto, con input di soli valori minimi negativi (`-128`).

  Questi test sono progettati per portare il `result_buffer_reg` ai suoi valori minimi e massimi possibili prima della normalizzazione, validando la scelta conservativa di utilizzare 24 bit per il buffer e verificando il corretto funzionamento del meccanismo di saturazione dell'output.

Il superamento di questo test bench dimostra un'elevata robustezza e aderenza del design alle specifiche, anche in condizioni operative non ideali.

4 Conclusioni

Il progetto descritto in questa relazione ha portato al completamento del design, dell'implementazione e della verifica del modulo VHDL descritto nella specifica.

L'architettura adottata, basata su una netta separazione tra un'unità di controllo a stati finiti (FSM) e logica combinatoria, rende l'architettura scalabile e di facile comprensione.

Dal punto di vista implementativo, i risultati della sintesi su FPGA hanno evidenziato un utilizzo ridotto delle risorse e il pieno rispetto dei vincoli di tempo, con una frequenza massima operativa ben superiore a quella richiesta.

In sintesi, il progetto ha soddisfatto tutti i requisiti funzionali, di robustezza e di performance. Il modulo finale non solo opera correttamente secondo il protocollo specificato, ma lo fa in modo efficiente.